

Итоговая сводка по реализации MNT4-753 pairing verification

Краткий отчет для обсуждения результатов работы

1 Исходная проблема

Цель работы — понять, можно ли построить более дешевую основу для будущих folding-схем на циклических MNT-кривых. В существующих подходах, например Sonobe/CycleFold, высокая стоимость часто появляется не из-за самого пользовательского утверждения, а из-за переноса арифметики одной кривой в поле другой кривой. Такая арифметика называется non-native: элемент большого поля разбивается на несколько частей, а каждое умножение и каждая редукция превращаются в большое число ограничений.

В EVM есть предкомпилированные контракты для BN254, но нет аналогичной поддержки для MNT4-753/MNT6-753. Поэтому MNT4-сопряжение можно либо вычислять вручную в Solidity/Yul, либо переносить тяжелые вычисления за пределы блокчейна и проверять на-chain только компактное утверждение о корректности.

2 Практический смысл работы

Практический смысл работы связан с задачей проверки длинных цепочек вычислений. В блокчейн-системах часто нужно доказать не один маленький факт, а длинную историю: что много транзакций обработаны корректно, что последовательность состояний получена по правилам протокола, что несколько доказательств действительно объединены в одно итоговое утверждение. Если проверять каждый шаг отдельно, стоимость растет линейно с длиной цепочки и быстро становится неприемлемой.

Простой пример — gollup. Пусть за один интервал времени вне основной сети обработано 10,000 транзакций. Наивный вариант — заново проверять в Ethereum каждую транзакцию или каждое изменение состояния. Практический вариант — построить одно доказательство того, что начальное состояние S_0 корректно перешло в состояние S_1 после всех 10,000 операций. Тогда on-chain контракту достаточно проверить короткий proof и новый state root, а не всю историю операций.

Второй пример — цепочка доказательств. Допустим, каждый час система выпускает proof корректности очередного batch-а транзакций. Через сутки появляется 24 отдельных доказательства. Рекурсивная схема позволяет построить новое доказательство, которое утверждает: первый proof корректен, второй proof корректен, и так далее, а итоговое состояние получено после всех 24 batch-ей. В результате контракт проверяет один proof вместо двадцати четырех. Именно это и называется сжатием цепочки доказательств.

Третий пример — мост между сетями. Мосту нужно убедить сеть B , что в сети A произошло событие: например, токены были заблокированы или сообщение было включено в состояние. Если передавать и проверять всю историю блоков, это

дорого. Если же периодически строить доказательства корректности обновления состояния сети A , то в сети B можно проверять только компактное итоговое доказательство. Здесь рекурсия нужна, чтобы не хранить и не проверять длинную историю обновлений.

Четвертый пример — приватный протокол. Пользователь может доказать, что он выполнил длинное вычисление или обладает правом на действие, не раскрывая все входные данные. Если таких действий много, отдельная проверка каждого proof становится дорогой. Рекурсивное или folding-доказательство позволяет объединить много маленьких утверждений в одно итоговое утверждение, сохранив приватность и уменьшив on-chain стоимость.

Рекурсивные доказательства решают общую проблему так: новое доказательство подтверждает не только очередной шаг вычисления, но и корректность предыдущего доказательства. Так получается цепочка доказательств, которую можно сжать до одного короткого финального proof. Для практики это важно в rollup-системах, мостах, проверке больших batch-вычислений, приватных протоколах и любых задачах, где on-chain нужно хранить и проверять только небольшой итог, а не всю историю вычислений.

MNT4/MNT6-кривые интересны тем, что образуют цикл полей: арифметика одной кривой естественно совместима со скалярным полем другой. Это делает их удобным объектом для рекурсии. Но у MNT4-753 есть практическая проблема: в стандартной EVM нет дешевой предкомпилированной поддержки для ее арифметики, а поле слишком большое для одного машинного слова. Поэтому без специальной реализации MNT4-сопряжения оказывается слишком дорогим.

Именно здесь появляется задача работы. Нужно понять, насколько далеко можно продвинуться без изменения EVM: реализовать длинную арифметику в Solidity/Yul, измерить полный on-chain вариант, применить оптимизации из литературы и проверить, дает ли переход к off-chain подготовке и on-chain проверке реальную экономию. Поэтому работа актуальна не как изолированная оптимизация одного контракта, а как исследование базового слоя для будущих дешевых рекурсивных и folding-схем.

3 Что было реализовано и измерено

В работе проверялись не отдельные эвристики, а несколько последовательных уровней реализации.

1. Оптимизированная длинная арифметика для MNT4-753 в Solidity/Yul.
2. Полное on-chain вычисление MNT4-сопряжения как контрольная реализация.
3. Вариант по статье *On Proving Pairings* (ePrint 2024/640): prepared-линии, проверка уравнения сопряжений и замена полной финальной экспоненты более короткой проверкой.
4. Исследование алгоритмов арифметики: Montgomery/Barrett, CIOS/FIOS/Comba, Karatsuba, lazy-reduction варианты и fused line-evaluation hot path.

5. Формальная оценка сложности: из стоимости операций в F_q , F_{q^2} , F_{q^4} выводится стоимость цикла Миллера, финальной экспоненты и verifier-сценариев.
6. Исследование альтернатив из статьи ePrint 2024/1627 и начало прототипа lollirop-305 как более легкой 2-limb арифметики.

Два отдельных документа фиксируют результаты по арифметике и сложности: ARITHMETIC_ALGORITHM_STUDY_RU.md и ALGORITHM_COMPLEXITY_ESTIMATES_RU.md.

4 Библиотека длинной арифметики в Solidity/Yul

Базовое поле MNT4-753 имеет размер около 753 бит, поэтому один элемент поля хранится в трех 256-битных словах EVM. Для него реализованы умножение, возведение в квадрат, сложение, вычитание и редукция. Над базовым полем построены расширения F_{q^2} и F_{q^4} , которые нужны для вычисления цикла Миллера и финальной экспоненты.

Рассматривались несколько вариантов реализации. Montgomery/CIOS оказался лучшим рабочим путем для длинных цепочек умножений. Barrett, FIOS и Comba/SOS были реализованы рядом как отрицательные сравнения: они корректны, но в EVM дают заметно большую стоимость из-за дополнительных переносов, промежуточных массивов и редукций.

Таблица 1: Стоимость базовых операций MNT4-753

Операция	Gas/op
Умножение в F_q	2,959
Возведение в квадрат в F_q	2,947
Умножение в F_{q^2}	11,877
Возведение в квадрат в F_{q^2}	10,592
Умножение в F_{q^4}	42,764
Возведение в квадрат в F_{q^4}	36,606

Эти значения объясняют порядок итоговых затрат: цикл Миллера и финальная экспонента состоят из сотен операций в F_{q^4} , поэтому даже оптимизированная реализация остается дорогой.

5 Формальная часть и оценки сложности

В теоретической части были оформлены утверждения, необходимые для корректной реализации:

1. модель полей, башни расширений и групп для MNT4-753;
2. описание ate-сопряжения через цикл Миллера и финальную экспоненту;
3. доказательство допустимости удаления знаменателя в вычислении функции Миллера;

4. обоснование prepared/sparse линий: коэффициенты можно готовить заранее, но on-chain код должен применять их к точкам и проверять итоговое отношение;
5. модель стоимости полного on-chain вычисления и verify-подходов.

Документ `ALGORITHM_COMPLEXITY_ESTIMATES_RU.md` отдельно показывает, как из цен $M_1, S_1, M_2, S_2, M_4, S_4$ получается стоимость более крупных блоков. Например, умножение в F_{q^2} реализуется через Karatsuba примерно как $3M_1 + O(A_1)$, а умножение в F_{q^4} — как $3M_2 + O(A_2)$. Поэтому оптимизация базовой арифметики напрямую влияет на стоимость всего сопряжения.

6 Полное on-chain вычисление

Полная реализация нужна как контрольная точка: она показывает, что будет, если считать MNT4-сопряжения в EVM без предкомпилированной поддержки. Такой вариант математически корректен и полезен для проверки других реализаций, но практически слишком дорог.

Таблица 2: Полное on-chain вычисление

Сценарий	Gas
Полное on-chain вычисление MNT4-сопряжения	259,535,379

Главная причина высокой стоимости — не одна отдельная операция, а вся цепочка: арифметика в F_{q^4} , сотни шагов цикла Миллера и финальная экспонента.

7 Direct residue verifier по статье ePrint 2024/640

В следующем варианте использовалась идея статьи *On Proving Pairings*: вместо проверки одного значения $Y = e(P, Q)$ удобнее проверять уравнение сопряжений

$$e(P, Q) \cdot e(-R, S) = 1.$$

Off-chain часть готовит коэффициенты линий Миллера и witness для проверки финальной экспоненты. On-chain контракт применяет prepared-линии к точкам, проходит цикл Миллера и вместо полной финальной экспоненты проверяет более короткое отношение с witness-параметром c . В оптимизированной реализации это дает экономию, но не устраняет основную стоимость цикла Миллера.

Сравнение нужно читать поэтапно. Переход от полного on-chain вычисления к prepared sparse-линиям резко снижает стоимость, потому что контракт больше не строит линии Миллера с нуля. Затем сравнение 115,997,313 и 93,879,746 показывает эффект оптимизации финальной экспоненты:

$$115,997,313 - 93,879,746 = 22,117,567 \text{ gas.}$$

Таблица 3: Сравнение этапов вычисления

Сценарий	Что меняется	Gas
Полное on-chain вычисление	линии и все вычисления считаются on-chain	259,535,379
Цикл Миллера для equation	без финальной экспоненты	87,747,219
Equation с полной финальной экспонентой	обычная FE	115,997,313
Equation с residue-проверкой	FE заменена witness-проверкой	93,879,746

Корректная формулировка вывода такая: оптимизация финальной экспоненты действительно уменьшает стоимость, но итоговый verifier все еще дорогой, потому что почти вся оставшаяся стоимость сидит в цикле Миллера. Сам цикл Миллера для equation стоит около

87,747,219 gas.

Следовательно, для практического результата нужно оптимизировать не только финальную экспоненту, но и проверку самого цикла Миллера.

8 Почему polynomial check не стал простым финальным решением

Статья ePrint 2024/640 предлагает не исполнять все умножения цикла Миллера on-chain. Вместо этого prover строит многочлены-свидетельства для последовательности вычислений, коммитится к ним, а verifier проверяет объединенное полиномиальное равенство в случайной точке. В упрощенном виде идея такая: если все переходы имеют вид

$$c_i = a_i \cdot b_i,$$

то prover представляет значения как многочлены $a(X)$, $b(X)$, $c(X)$, а verifier проверяет, что

$$a(\alpha)b(\alpha) - c(\alpha) = Z(\alpha)Q(\alpha)$$

для случайного α . Здесь $Z(X)$ зануляется на точках домена, а $Q(X)$ — частный многочлен. Если prover подделал хотя бы часть вычислений, то равенство с высокой вероятностью нарушится.

Для MNT4-753 в EVM у этого подхода есть два естественных пути, и оба пока не стали простым production-решением.

1. KZG over BN254. Проверка открытия короткая и удобная для EVM, потому что BN254 поддерживается предкомпилированными контрактами. Но сами MNT4-элементы имеют размер 753 бита, а поле BN254 существенно меньше. Поэтому

арифметика MNT4 внутри BN254-доказательства становится non-native и снова дает миллионы ограничений. Это близко к той же проблеме, которая возникает в Sonobe/CycleFold.

2. Merkle/FRI. Этот вариант не требует KZG и может быть ближе к нативному полю, но доказательство становится большим. Один элемент MNT4-поля занимает около 96 байт, а для открытия многочленов нужны значения, Merkle-пути и дополнительные данные. Поэтому стоимость переносится из вычислений в call-data и хеширование.

Итог: polynomial check является правильным направлением для дальнейшей работы, но его нельзя просто добавить как маленькую on-chain функцию. Нужно отдельно выбирать схему commitments/openings и доказывать, что она не возвращает нас к той же проблеме non-native арифметики или слишком большого calldata.

9 Исследование статьи ePrint 2024/1627 и lollipop-305

По замечанию о более совершенных техниках была отдельно изучена статья ePrint 2024/1627. Ее идея состоит в построении циклов supersingular elliptic curves для pairing-based proof systems. Практический смысл для данной работы такой: если найти кривую с меньшим базовым полем и удобным циклом, то можно уменьшить стоимость базовой арифметики и потенциально сделать future folding дешевле.

Для проверки этой гипотезы был начат исследовательский прототип lollipop305_research. В нем реализована 2-limb арифметика для примера lollipop-305: базовое поле, F_{p^2} , F_{p^4} , Montgomery/CIOS, сравнение с Comba/FIOS/Barrett и Foundry gas-тесты.

Таблица 4: lollipop-305 против MNT4-753

Операция	lollipop-305	MNT4-753	Выигрыш
F_p mul	2,081	3,287	1.58x
F_p square	1,926	2,939	1.53x
F_{p^2} mul	7,361	11,877	1.61x
F_{p^2} square	6,289	10,592	1.68x
F_{p^4} mul	24,243	42,764	1.76x
F_{p^4} square	22,028	36,606	1.66x

Результат положительный, но осторожный: 2-limb поле действительно дешевле, в среднем примерно в полтора раза. Это показывает перспективность направления, но не доказывает готовность production-кривой. Использованный lollipop-305 является исследовательским примером с недостаточной криптографической стойкостью. Более безопасные кривые из этого семейства имеют больший размер поля, иногда больше MNT4-753, поэтому их on-chain арифметика может снова стать дорогой.

На этом этапе направление lollipop остановлено на уровне арифметики. Для продолжения нужно отдельно решить математическую задачу: строго зафиксировать вторую группу, twist, генераторы и non-degenerate pairing для выбранных параметров.

10 Итоговое состояние

К текущему моменту получены следующие результаты.

1. Построена и протестирована оптимизированная Solidity/Yul арифметика для MNT4-753.
2. Реализованы и сравнены альтернативные алгоритмы умножения и редукции; выбран Montgomery/CIOS как лучший вариант для EVM.
3. Составлена модель сложности, связывающая стоимость базовых операций с полной стоимостью цикла Миллера, финальной экспоненты и verifier-сценариев.
4. Реализовано полное on-chain вычисление MNT4-сопряжения и показана его непрактичность по gas.
5. Реализован direct residue verifier по статье ePrint 2024/640; оптимизация финальной экспоненты дает экономию около 22.1 млн gas, но не решает стоимость цикла Миллера.
6. Проанализирован polynomial-check путь: он является естественным продолжением статьи, но требует отдельного commitments/openings слоя.
7. Исследована статья ePrint 2024/1627 и начат lollipop-305 прототип; арифметика дешевле MNT4-753 примерно в 1.5–1.8 раза, но production-вариант требует отдельного выбора безопасной кривой и полной pairing-спецификации.